



MEVN

Chapitre 11 :

Notion sur les composants et les props

Madame FANOMEZANA Mihajaso Léa
2026

SFC

- SFC : Single File Component (Composant à fichier unique)
- Un composant SFC est un fichier avec l'extension **.vue** qui regroupe dans un seul fichier :
 - **Le template (HTML)** : ce qui est affiché à l'écran.
 - **Le script (JavaScript)** : la logique du composant.
 - **Le style (CSS)** : la mise en forme.

<template>...</template>

<script>...</script>

<style>...</style>

Notre première page SFC

```
<template>
  <h1>{{ message }}</h1>
</template>

<script>
export default {
  data() {
    return {
      message: "This is some text",
    };
  },
};
</script>
<style></style>
```

Les composants Vue (Component)

Le concept des Composants se repose sur la subdivision de la page web en plusieurs parties afin de rendre facile le développement de l'application

- Facile
- Bien ordonné
- Indépendant les uns des autres
- Réutilisable
- Scalable

Création d'un composant

- Création d'un dossier nommé *Components* dans *src*
- Création d'un composant *FruitItem.vue*

```
<template>
  <div>
    <h2>{{ name }}</h2>
    <p>{{ message }}</p>
  </div>
</template>
```

```
<script>
export default {
  data() {
    return {
      name: "Pomme",
      message: "Voici ma pomme",
    };
  },
};
</script>
<style></style>
```

Ajout du premier composant

- Ajout de l'import dans main.js

```
import FruitItem from './components/FruitItem.vue'
```

- Puis, on personnalise le nom du composant en ajoutant dans main.js

```
app.component('FruitItem', FruitItem)
```

OU on met directement la ligne suivante dans la balise <script> de la page où on l'utilise (composant local) :

```
<script setup> import FruitItem from './components/FruitItem.vue'</script>
```

- Pour l'utilisation du composant dans App.vue :

```
<FruitItem></FruitItem> ou <FruitItem />
```

Composants et événements

FruitItem.vue

```
<template>
  <div v-on:click="countClicks">
    <h2>{{ name }}</h2>
    <p>{{ message }}</p>
    <p id="red">
      vous avez cliqué sur moi {{ clicks }} fois
    </p>
  </div>
</template>

<style>
#red {
  font-weight: bold;
  color: rgb(144, 12, 12);
}
</style>
```

```
<script>
export default {
  data() {
    return {
      name: 'pomme',
      message: 'voici ma pomme',
      clicks: 0,
    }
  },
  methods: {
    countClicks() {
      this.clicks++
    },
  },
}
</script>
```

Props

Props est une option de configuration dans Vue qui permet de transmettre des données vers un composant via des attributs personnalisés de la balise du composant

FruitItem.vue

```
<template>
  <h2>{{ nomFruit }}</h2>
</template>

<script>
export default {
  props: ['nomFruit'],
}
</script>
```

App.vue

```
<script setup>
import FruitItem from './components/FruitItem.vue'
</script>

<template>
  <h1>Fruit</h1>
  <FruitItem nomFruit="Pomme" />
  <FruitItem nomFruit="Ananas" />
</template>
```


Interface Props

- Le code ci-dessus ne permet pas de savoir les autres caractéristiques requises pour la propriété `nomFruit` à savoir le type de données, si la propriété est obligatoire ou non,...
- Pour remédier à cela, la solution que Vue propose est le props en tant qu'objet et le props requis

Props en tant qu'objet

```
<script>
export default {
  // props:
  ['nomFruit', estSucre],
  props: {
    nomFruit: String,
    estSucre: Boolean,
  },
}
</script>
```

- En tant qu'objet, on peut définir **le type de données** de chaque propriété en plus de son nom
- Avec des propriétés définies de cette manière, d'autres personnes dans une équipe peuvent regarder à l'intérieur **FruitItem.vue** et voir facilement ce que le composant attend.

FruitItem.vue

```
<template>
  <h2>{{ nomFruit }}</h2>
  <p v-show="estSucre">
    Oui c'est sucre pas acide
  </p>
</template>

<script>
export default {
  // props: ['nomFruit'],
  props: {
    nomFruit: String,
    estSucre: Boolean,
  },
}
```

App.vue

```
<script setup>
import FruitItem from
'./components/FruitItem.vue'
</script>

<template>
  <h1>Fruit</h1>

  <FruitItem
    nomFruit="Pomme"
    v-bind:estSucre="true" />
    <FruitItem nomFruit="Ananas" />
</template>
```

« estSucre : Boolean » indique à Vue que la **prop devrait être un booléen**, mais cela ne transforme pas automatiquement toutes les valeurs reçues en booléen → v-bind

Props requis

- Pour indiquer à Vue qu'une propriété est obligatoire, il faut la définir comme un objet requis.
- Puis mettre « require : true »

```
<script>
export default {
  // props: ['nomFruit'],
  props: {
    fruit: {
      type: String,
      require: true,
    },
    estSucre: Boolean,
  },
}
</script>
```

Props par défaut

- Pour définir une valeur par défaut d'un props, on utilise « default : 'la valeur par défaut' »

```
<script>
export default {
  // props: ['nomFruit'],
  props: {
    fruit: {
      type: String,
      require: true,
    },
    estSucre: {
      type: Boolean,
      default: true,
    },
  },
},
}
</script>
```

Fonction de validation de props

- La fonction de validation détermine si la valeur attendue par un props valide ou non
- C'est une fonction qui doit retourner par true ou false
- En cas d'une valeur invalide du props, un avertissement dans la console du navigateur lors de l'exécution de la page en mode développeur est généré
- La fonction de validation assure que les composants sont utilisé correctement

FruitItem.vue

```
<script>
export default {
  // props: ['nomFruit'],
  props: {
    nomFruit: {
      type: String,
      require: true,
      validator: function (value) {
        if (value.length < 6) {
          return true
        } else {
          return false
        }
      },
    },
    estSucre: Boolean,
  },
}
</script>
```

Exemple : on peut supposer que le nom d'un fruit ne doit pas avoir une longueur supérieure à 20.

App.vue

```
<template>
  <h1>Fruit</h1>

  <FruitItem
    nomFruit="pomme"
    v-bind:estSucre="true" />
  <FruitItem
    nomFruit="Pastèque" />
</template>
```

Modification d'un props

- Lorsqu'un composant est créé dans l'élément parent, il est impossible de modifier la valeur de la propriété reçue de l'élément enfant.
- Par conséquent, à l'intérieur de l'élément enfant, **FruitItem.vue** on ne peut pas modifier la valeur de la propriété « estSucre » qui lui est transmise **App.vue**.
- **TP de recherche : Proposez une solution avec un exemple**

Une solution

FruitItem.vue

```
<template>
  <div>
    <h2>
      {{ foodName }}
    </h2>
    <p>{{ foodDesc }}</p>
    
    <button v-
on:click="toggleFavorite">Favorite</bu
tton>
  </div>
</template>
```

```
<script>
export default {
  props: [
    'foodName',
    'foodDesc',
    'isFavorite'
  ],
  data() {
    return {
      foodIsFavorite: this.isFavorite,
    }
  },
  methods: {
    toggleFavorite() {
      this.foodIsFavorite =
!this.foodIsFavorite
    },
  },
}
</script>
```

```
<script setup>
import FruitItem from './components/FruitItem.vue'
</script>
<template>
  <h1>Nourriture</h1>
  <div id="wrapper">
    <FruitItem food-name="Pomme" food-desc="Pomme" v-bind:is-favorite="true" />
    <FruitItem food-name="Banane" food-desc="Banane " v-bind:is-favorite="false" />
  </div>
</template>

<style>
#wrapper {
  display: flex;
  flex-wrap: wrap;
}
#wrapper > div {
  border: dashed black 1px;
  flex-basis: 120px;
  margin: 10px;
  padding: 10px;
  background-color: rgb(144, 202, 238);
}
</style>
```